

Cuestionario de estudio

Java 17 & 21 — Certificación Oracle OCP

Basado en los temas del curso "Curso Certificación Java 17 & 21"

Canal Hello World Java — Miguel Rugerio

85 preguntas de opción múltiple organizadas en 24 categorías, alineadas con los objetivos del examen Oracle Certified Professional Java SE 17/21 Developer (1Z0-829).

Cómo usar esta guía:

- Responde primero todas las preguntas de una categoría sin mirar las respuestas.
- Anota tus respuestas (A, B, C o D) en una hoja aparte.
- Al final del documento encontrarás la sección de Respuestas y Explicaciones, organizada igual que las preguntas.
- Repite las categorías donde falles más de una pregunta hasta dominar el 90% o más.

Índice de categorías

1. Fundamentos JVM/JDK — 3 preguntas
2. Primitivos, casting y var — 4 preguntas
3. Operadores y control de flujo — 3 preguntas
4. Arrays y varargs — 3 preguntas
5. Strings y bloques de texto — 4 preguntas
6. POO: clases, constructores y encapsulamiento — 4 preguntas
7. Herencia y polimorfismo — 4 preguntas
8. Interfaces y clases abstractas — 4 preguntas
9. Enums — 3 preguntas
10. Records — 3 preguntas
11. Sealed classes y pattern matching — 4 preguntas
12. Generics y covarianza — 4 preguntas
13. Comparable y Comparator — 3 preguntas
14. Lambdas e interfaces funcionales — 4 preguntas
15. Method references — 3 preguntas
16. Streams API — 6 preguntas
17. Optional — 3 preguntas
18. Collections Framework — 5 preguntas
19. Concurrencia y virtual threads — 5 preguntas
20. Manejo de excepciones — 4 preguntas
21. BigDecimal y precisión numérica — 3 preguntas
22. Internacionalización (Locale, ResourceBundle) — 3 preguntas
23. Patrones de diseño (Strategy) — 3 preguntas

Fundamentos JVM/JDK

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

- A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode
- B) JRE es para desarrollar, JDK es solo para ejecutar programas ya compilados
- C) La JVM compila el código fuente .java directamente a código máquina nativo sin bytecode intermedio
- D) JDK y JRE son sinónimos y ambos requieren instalar la JVM por separado

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

- A) .class
- B) .jar
- C) .exe
- D) .bytecode

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

- A) java MiArchivo.java
- B) javac -run MiArchivo.java
- C) jshell MiArchivo.java --exec
- D) java --compile MiArchivo.java

Primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

- A) byte, short, int, long, float, double, char, boolean
- B) byte, short, int, long, float, double, String, boolean
- C) int, long, float, double, char, boolean, Integer, Double
- D) byte, int, long, double, char, boolean, var, Object

2. ¿Qué imprime este código?

int x = 130;

byte b = (byte) x;

System.out.println(b);

```
int x = 130;
byte b = (byte) x;
System.out.println(b);
```

- A) -126
- B) 130
- C) Error de compilación
- D) 0

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

- A) var x; (sin inicializador)
- B) var lista = new ArrayList<String>();
- C) var texto = "hola";
- D) var numero = 10;

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

- A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)
- B) Sí, siempre que el campo sea private
- C) Sí, pero solo si el campo es static final
- D) Sí, en cualquier contexto incluyendo parámetros de métodos

Operadores y control de flujo

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;
```

```
String r = switch(dia){
```

```
case 1,7 -> "Fin de semana";
```

```
default -> "Entre semana";
```

```
};
```

```
System.out.println(r);
```

```
int dia = 3;
String r = switch(dia){
case 1, 7 -> "Fin de semana";
default -> "Entre semana";
};
System.out.println(r);
```

- A) Entre semana
- B) Fin de semana
- C) Error de compilación
- D) 3

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

- A) yield
- B) return
- C) break
- D) value

3. ¿Cuál es el resultado de: `System.out.println(5 / 2);` y luego `System.out.println(5.0 / 2);`?

- A) 2 y 2.5
- B) 2.5 y 2.5
- C) 2 y 2
- D) 2.5 y 2

Arrays y varargs

1. ¿Cuál declaración de un método varargs es válida?

- A) void metodo(String... nombres)
- B) void metodo(String nombres...)
- C) void metodo(...String nombres)
- D) void metodo(String[]... nombres, int x)

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

- A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs
- B) Error de compilación por ambigüedad siempre
- C) Siempre se ejecuta la versión varargs
- D) Solo compila si varargs se declara primero en el archivo

3. `int[] a = new int[3];` ¿cuál es el valor por defecto de cada elemento?

- A) 0
- B) null
- C) Error de compilación, hay que inicializar
- D) -1

Strings y bloques de texto

1. ¿Por qué se dice que String es inmutable en Java?

- A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto
- B) Porque String no permite ser referenciado por más de una variable
- C) Porque el compilador prohíbe declarar más de un String por clase
- D) Porque String usa arrays mutables internamente sincronizados

2. ¿Qué imprime?

String a = "hola";

String b = "hola";

System.out.println(a == b);

```
String a = "hola";  
String b = "hola";  
System.out.println(a == b);
```

- A) true
- B) false
- C) Error de compilación
- D) Depende del JIT

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

- A) Triple comillas dobles """" seguidas de salto de línea
- B) Comillas simples triples '''
- C) Corchetes angulares <"">
- D) Prefijo tb" seguido de comillas dobles

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

- A) equals()
- B) ==
- C) compareRef()
- D) same()

POO: clases, constructores y encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

- A) El compilador genera automáticamente un constructor por defecto sin argumentos
- B) La clase no puede ser instanciada nunca
- C) Java lanza una excepción en tiempo de ejecución al usar new
- D) Se genera un constructor con todos los atributos como parámetros

2. ¿Qué significa que un atributo sea 'static'?

- A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual
- B) No puede modificarse nunca después de asignarse
- C) Solo puede accederse desde el método main
- D) Se destruye automáticamente al finalizar cada método

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

- A) Impedir que la variable sea reasignada después de su primera asignación
- B) Hacer que la variable sea accesible desde cualquier paquete
- C) Convertir la variable en static automáticamente
- D) Permitir que la variable se sobrescriba en subclases

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

- A) No, una clase final no puede tener subclases
- B) Sí, siempre
- C) Solo si está en el mismo paquete
- D) Solo si implementa una interfaz

Herencia y polimorfismo

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

- A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación
- B) Definir varios métodos con el mismo nombre pero distintos parámetros en la misma clase
- C) Declarar un método como private en una subclase
- D) Llamar a un método estático desde una instancia

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

- A) super
- B) this
- C) parent
- D) extends

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }
```

```
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }
```

```
Animal a = new Perro();
```

```
a.sonido();
```

¿Qué se imprime?

```
class Animal { void sonido(){ System.out.println("..."); } }  
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }  
Animal a = new Perro();  
a.sonido();
```

- A) Guau
- B) ...
- C) Error de compilación
- D) Guau seguido de ...

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

- A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero SÍ permite implementar múltiples interfaces
- B) Verdadero, ambas cosas son posibles sin restricción
- C) Falso, tampoco permite implementar múltiples interfaces
- D) Verdadero, pero solo desde Java 21

Interfaces y clases abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

- A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir
- B) Hacer que el método sea abstracto obligatoriamente
- C) Convertir la interfaz en una clase abstracta automáticamente
- D) Ejecutarse solo si la clase implementadora es static

2. ¿Puede una clase abstracta tener constructores?

- A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas
- B) No, nunca
- C) Solo si es final
- D) Solo si no tiene métodos abstractos

3. ¿Qué es una clase anónima en Java?

- A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual
- B) Una clase que no puede tener métodos
- C) Un tipo de enum sin constantes
- D) Una clase que solo puede usarse dentro de records

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

- A) Exactamente uno (además de métodos default/static)
- B) Cero
- C) Máximo dos
- D) Ilimitados

Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

- A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces
- B) No, un enum solo puede contener constantes simples
- C) Solo puede tener métodos static
- D) Solo puede implementar una interfaz si no tiene constructor

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

- A) ordinal()
- B) position()
- C) index()
- D) value()

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

- A) Sí, solo se usa el nombre de la constante (ej. case LUNES ->)
- B) No, siempre hay que escribir DiaSemana.LUNES
- C) Solo si el enum es public
- D) Solo en switch clásico, no en switch expression

Records

1. ¿Qué genera automáticamente el compilador al declarar un record?

- A) Constructor canónico, métodos de acceso (accessors) con el nombre del componente, equals(), hashCode() y toString()
- B) Solo el método toString()
- C) Getters con prefijo 'get' como en un JavaBean tradicional
- D) Setters para modificar cada componente

2. ¿Son mutables los componentes de un record?

- A) No, todos los campos de un record son implícitamente final e inmutables
- B) Sí, se pueden reasignar con setters generados
- C) Solo si se declaran como var
- D) Depende de si el record implementa Serializable

3. ¿Qué es un 'constructor compacto' en un record?

- A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita
- B) Un constructor que no acepta parámetros
- C) Un constructor privado obligatorio en todo record
- D) Un método estático que reemplaza al constructor

Sealed classes y pattern matching

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

- A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'
- B) Para hacerla immutable automáticamente como un record
- C) Para prohibir que tenga métodos abstractos
- D) Para convertirla automáticamente en final

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

- A) final, sealed o non-sealed
- B) static
- C) abstract siempre
- D) private

3. ¿Qué permite el pattern matching for instanceof (Java 16+)?

- A) Combinar la comprobación instanceof con la asignación a una variable en una sola expresión, evitando el cast manual: `if (obj instanceof String s) {...}`
- B) Comparar dos instancias por referencia
- C) Convertir automáticamente cualquier objeto a String
- D) Verificar solo si un objeto es null

4. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

- A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. `case Punto(int x, int y) -> ...`)
- B) Permite modificar los valores del record dentro del switch
- C) Obliga a que el record implemente Comparable
- D) Solo funciona con enums, no con records

Generics y covarianza

1. ¿Qué problema resuelven los Generics en Java?

- A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando ClassCastException en tiempo de ejecución y castings manuales
- B) Permiten que una variable cambie de tipo dinámicamente en tiempo de ejecución
- C) Eliminan la necesidad de interfaces
- D) Sirven solo para arrays primitivos

2. ¿Qué significa el comodín '?' extends Number' en una declaración de generics?

- A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')
- B) Acepta solo el tipo Number exacto
- C) Acepta cualquier tipo excepto Number
- D) Permite modificar la colección añadiendo elementos de cualquier subtipo libremente

3. ¿Por qué List<String> NO es un subtipo de List<Object> en Java (a diferencia de los arrays)?

- A) Porque los generics son invariantes: aunque String sea subtipo de Object, List<String> y List<Object> son tipos distintos sin relación de subtipo entre sí
- B) Porque List<String> no puede contener objetos de tipo Object
- C) Porque los generics no soportan herencia en absoluto
- D) Porque solo los arrays son válidos en Java, List<> es obsoleto

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

- A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u Object) en el bytecode
- B) Un proceso que elimina variables no usadas del código fuente
- C) Una técnica para borrar clases genéricas del classpath
- D) Un mecanismo del recolector de basura para objetos genéricos

Comparable y Comparator

1. ¿Cuál es la diferencia principal entre Comparable y Comparator?

- A) Comparable se implementa en la propia clase para definir su orden natural (compareTo); Comparator se define externamente y permite múltiples estrategias de ordenación (compare)
- B) Comparable es para colecciones y Comparator solo para arrays
- C) Comparator solo funciona con tipos primitivos
- D) No hay diferencia, son sinónimos desde Java 17

2. ¿Qué valor devuelve compareTo() si el objeto actual es 'mayor' que el que se compara?

- A) Un valor positivo
- B) Un valor negativo
- C) Siempre 1 exactamente
- D) 0

3. ¿Cómo se encadenan múltiples criterios de comparación usando Comparator (por ejemplo, por apellido y luego por nombre)?

- A) Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)
- B) Comparator.and(Persona::getApellido, Persona::getNombre)
- C) Comparator.merge(apellido, nombre)
- D) No es posible encadenar comparadores en Java

Lambdas e interfaces funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

- A) (a, b) -> a + b
- B) function(a, b) => a + b
- C) lambda(a, b) { return a + b; }
- D) (a, b) => { a + b }

2. ¿Qué interfaz funcional del paquete java.util.function representa una operación que recibe un argumento y devuelve un boolean?

- A) Predicate<T>
- B) Function<T,R>
- C) Supplier<T>
- D) Consumer<T>

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

- A) Supplier<T>
- B) Consumer<T>
- C) Function<T,R>
- D) BiFunction<T,U,R>

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

- A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas
- B) Sí, puede modificar libremente cualquier variable local externa
- C) Solo si la variable es static
- D) Solo dentro de un Comparator

Method references

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

- A) Clase::metodoEstatico
- B) Clase->metodoEstatico
- C) Clase.metodoEstatico()
- D) new Clase::metodo

2. ¿Qué representa la referencia a método `String::toUpperCase` usada como `Function<String,String>`?

- A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a `s -> s.toUpperCase()`
- B) Una referencia a un constructor
- C) Una referencia a un método estático de la clase `String`
- D) Un error de compilación porque `toUpperCase` no es estático

3. ¿Cómo se escribe una referencia a constructor para crear instancias de `ArrayList` mediante un `Supplier`?

- A) `ArrayList::new`
- B) `new ArrayList()::create`
- C) `ArrayList.new()`
- D) `Supplier.of(ArrayList)`

Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

- A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez
- B) Un stream modifica siempre la colección original
- C) Un stream se puede recorrer múltiples veces como una lista
- D) Un stream requiere que la colección esté ordenada previamente

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

- A) collect()
- B) filter()
- C) map()
- D) sorted()

3. ¿Qué hace el siguiente código?

List<String> nombres = List.of("Ana","Luis","Eva");

nombres.stream().filter(n -> n.length()>3).forEach(Svstem.out::println);

```
List<String> nombres = List.of("Ana", "Luis", "Eva");
nombres.stream()
    .filter(n -> n.length() > 3)
    .forEach(System.out::println);
```

- A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres
- B) Imprime los tres nombres
- C) No compila
- D) Imprime "Ana" y "Eva"

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

- A) java.util.stream.Collectors
- B) java.util.Collection
- C) java.util.Accumulator
- D) java.util.stream.Reducer

5. ¿Qué hace Collectors.groupingBy()?

- A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un Map<clave, List<elementos>>
- B) Ordena los elementos alfabéticamente
- C) Elimina duplicados del stream
- D) Convierte el stream en un array

6. ¿Cuál es la diferencia entre map() y flatMap() en Streams?

- A) map() transforma cada elemento en otro elemento uno a uno; flatMap() transforma cada elemento en un stream y luego 'aplana' todos esos streams en uno solo
- B) map() solo funciona con enteros y flatMap solo con Strings
- C) flatMap() es simplemente un alias de map()
- D) map() es una operación terminal y flatMap() es intermedia

Optional

1. ¿Cuál es el propósito principal de la clase `Optional<T>`?

- A) Representar explícitamente la posible ausencia de un valor, evitando `NullPointerException` y forzando un manejo explícito del caso 'vacío'
- B) Reemplazar por completo el uso de `null` en cualquier contexto, incluidos campos de clase
- C) Mejorar el rendimiento de las colecciones
- D) Servir como una nueva colección ordenada

2. ¿Qué hace `Optional.ofNullable(valor)` si 'valor' es `null`?

- A) Devuelve un `Optional` vacío (`Optional.empty()`) sin lanzar excepción
- B) Lanza `NullPointerException` inmediatamente
- C) Devuelve `null`
- D) No compila

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un `Optional` si está vacío?

- A) `optional.orElse(valorPorDefecto)`
- B) `optional.getDefault()`
- C) `optional.value()`
- D) `optional.orNull()`

Collections Framework

1. ¿Cuál es la diferencia principal entre List, Set y Map en el Collections Framework?

- A) List permite duplicados y mantiene orden de inserción; Set no permite duplicados; Map almacena pares clave-valor
- B) List no permite duplicados; Set sí los permite; Map es solo para arrays
- C) Los tres son idénticos, solo cambia el nombre
- D) Set mantiene siempre orden alfabético automáticamente y List nunca permite acceso por índice

2. ¿Qué implementación de List garantiza acceso rápido $O(1)$ por índice, pero inserciones/eliminaciones costosas en el medio?

- A) ArrayList
- B) LinkedList
- C) Vector solamente
- D) TreeSet

3. ¿Qué característica distingue a un TreeMap de un HashMap?

- A) TreeMap mantiene sus claves ordenadas (orden natural o Comparator); HashMap no garantiza ningún orden
- B) TreeMap permite claves duplicadas y HashMap no
- C) HashMap es más lento que TreeMap en todos los casos
- D) TreeMap no permite valores null nunca y HashMap sí siempre

4. ¿Qué devuelve List.of(1,2,3) en cuanto a mutabilidad?

- A) Una lista inmutable; intentar añadir o eliminar elementos lanza UnsupportedOperationException
- B) Una lista completamente mutable equivalente a ArrayList
- C) Un array primitivo int[]
- D) Un Set en lugar de una List

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un HashMap de forma consistente?

- A) equals() y hashCode() (contrato de Object)
- B) Comparable obligatoriamente
- C) Serializable obligatoriamente
- D) Cloneable obligatoriamente

Concurrencia y virtual threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

- A) Callable<V>
- B) Runnable
- C) Thread
- D) Executor

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

- A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida
- B) Solo permite ejecutar un hilo a la vez
- C) Reemplaza completamente la necesidad de sincronización
- D) Es exclusivo para aplicaciones de red

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

- A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O
- B) Hilos que solo pueden ejecutar código funcional (lambdas)
- C) Un reemplazo para las clases enum
- D) Hilos que se ejecutan exclusivamente en la GPU

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

- A) synchronized
- B) volatile
- C) static
- D) transient

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

- A) Executors.newVirtualThreadPerTaskExecutor()
- B) Executors.newFixedThreadPool(1)
- C) Thread.ofVirtual().createPool()
- D) new VirtualExecutorService()

Manejo de excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

- A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden Exception); las unchecked no obligan a manejo explícito (extienden RuntimeException)
- B) Las unchecked solo pueden lanzarse dentro de un try-catch
- C) Las checked no pueden ser capturadas nunca con catch
- D) No hay diferencia real, es solo una convención de nombres

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

- A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo System.exit() o error grave de JVM)
- B) Solo se ejecuta si no ocurrió ninguna excepción
- C) Solo se ejecuta si hubo una excepción no capturada
- D) Se ejecuta antes que el bloque try

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

- A) Cierra automáticamente los recursos que implementan AutoCloseable al finalizar el bloque try, sin necesidad de un finally explícito
- B) Permite capturar cualquier tipo de excepción sin declarar catch
- C) Elimina la necesidad de manejar excepciones por completo
- D) Solo funciona con archivos, no con conexiones de red o BD

4. ¿Qué clase es la superclase común de Exception y Error?

- A) Throwable
- B) RuntimeException
- C) Object
- D) IOException

BigDecimal y precisión numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

- A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria
- B) Porque double no puede almacenar números negativos
- C) Porque BigDecimal es más rápido en todas las operaciones aritméticas
- D) Porque double no permite más de 2 decimales

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

- A) `new BigDecimal("19.99")` usando un String, para evitar imprecisión del double
- B) `new BigDecimal(19.99)` usando el double directamente, siempre es exacto
- C) `BigDecimal.valueOf(19.99f)` usando float
- D) No es posible crear BigDecimal con decimales

3. ¿Por qué comparar dos BigDecimal con `equals()` puede dar resultados inesperados, por ejemplo entre `new BigDecimal("2.0")` y `new BigDecimal("2.00")`?

- A) Porque `equals()` considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar `compareTo()`
- B) Porque BigDecimal no sobrescribe `equals()`
- C) Porque BigDecimal siempre lanza excepción al comparar
- D) Porque `equals()` en BigDecimal compara referencias, no valores

Internacionalización (Locale, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

- A) `java.util.Locale`
- B) `java.util.Region`
- C) `java.util.Country`
- D) `java.text.Zone`

2. ¿Cómo se cargan mensajes traducidos según el idioma usando ResourceBundle?

- A) `ResourceBundle.getBundle("nombreBase", locale)` busca automáticamente el archivo `.properties` más específico que coincida con el `Locale` dado, con fallback al bundle por defecto
- B) Se debe escribir manualmente un `if-else` para cada idioma soportado
- C) `ResourceBundle` solo soporta inglés y español
- D) Los archivos `.properties` deben estar compilados como clases `.class`

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un Locale específico?

- A) `java.text.NumberFormat`
- B) `java.lang.NumberParser`
- C) `java.util.Formatter.Locale`
- D) `java.text.LocaleFormat`

Patrones de diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

- A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa
- B) Permite crear una única instancia global de una clase
- C) Convierte una interfaz incompatible en otra que el cliente espera
- D) Permite añadir responsabilidades a un objeto dinámicamente envolviéndolo

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

- A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia
- B) Los records
- C) Los enums simples
- D) Los bloques de texto

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

- A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)
- B) Hace que el código se ejecute más rápido siempre
- C) Elimina la necesidad de interfaces
- D) Convierte el código automáticamente en concurrente

Respuestas y explicaciones

Fundamentos JVM/JDK

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

Respuesta correcta: A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode

El JDK (Java Development Kit) contiene compilador, herramientas y el JRE. El JRE (Java Runtime Environment) trae la JVM y librerías estándar para ejecutar. La JVM interpreta/compila JIT el bytecode .class.

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

Respuesta correcta: A) .class

javac genera archivos .class que contienen bytecode, ejecutado luego por la JVM.

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

Respuesta correcta: A) java MiArchivo.java

Desde JEP 330 (Java 11) se puede ejecutar 'java MiArchivo.java' para lanzar un programa de un solo archivo fuente sin compilación explícita.

Primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

Respuesta correcta: A) byte, short, int, long, float, double, char, boolean

Los 8 tipos primitivos son: byte, short, int, long, float, double, char y boolean. String no es primitivo.

2. ¿Qué imprime este código?

```
int x = 130;  
byte b = (byte) x;  
System.out.println(b);
```

Respuesta correcta: A) -126

byte es de 8 bits (rango -128 a 127). 130 se desborda: $130 - 256 = -126$ debido al overflow al hacer narrowing cast.

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

Respuesta correcta: A) var x; (sin inicializador)

'var' requiere inicialización en la misma línea porque el compilador infiere el tipo a partir del valor asignado; no puede usarse sin inicializador ni como tipo de campo/parámetro.

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

Respuesta correcta: A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)

'var' (inferencia de tipo local, LVTI) solo aplica a variables locales, no a campos de clase, parámetros de método o tipos de retorno.

Operadores y control de flujo

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;  
String r = switch(dia){  
case 1,7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

Respuesta correcta: A) Entre semana

Con la nueva sintaxis de switch expression (flechas), múltiples valores se agrupan con comas. Como dia=3 no coincide con 1 ni 7, cae en default.

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

Respuesta correcta: A) yield

'yield' se usa dentro de un bloque {} de un switch expression para producir el valor resultante.

3. ¿Cuál es el resultado de: System.out.println(5 / 2); y luego System.out.println(5.0 / 2);?

Respuesta correcta: A) 2 y 2.5

La división entera (int/int) trunca el decimal, da 2. Si al menos un operando es double, el resultado es double, da 2.5.

Arrays y varargs

1. ¿Cuál declaración de un método varargs es válida?

Respuesta correcta: A) void metodo(String... nombres)

La sintaxis correcta de varargs es 'Tipo... nombre'. Solo puede haber un parámetro varargs y debe ser el último de la lista.

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

Respuesta correcta: A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs

La resolución de sobrecarga en Java prioriza la coincidencia sin varargs (aplicabilidad estricta) antes de considerar métodos varargs.

3. int[] a = new int[3]; ¿cuál es el valor por defecto de cada elemento?

Respuesta correcta: A) 0

Los arrays de tipos primitivos numéricos se inicializan por defecto a 0 (boolean a false, char a '\u0000', referencias a null).

Strings y bloques de texto

1. ¿Por qué se dice que String es inmutable en Java?

Respuesta correcta: A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto

Los objetos String son inmutables: cualquier método que 'modifica' un String (concat, replace, toUpperCase...) realmente crea y devuelve un nuevo objeto.

2. ¿Qué imprime?

```
String a = "hola";  
String b = "hola";  
System.out.println(a == b);
```

Respuesta correcta: A) true

Los literales String se almacenan en el String pool. Ambas variables referencian el mismo objeto interno, así que '==' (comparación de referencia) da true.

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

Respuesta correcta: A) Triple comillas dobles """" seguidas de salto de línea

Un text block comienza con tres comillas dobles y un salto de línea inmediatamente después, terminando también con tres comillas dobles.

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

Respuesta correcta: A) equals()

'equals()' compara el contenido carácter por carácter; '==' compara si ambas referencias apuntan al mismo objeto en memoria.

POO: clases, constructores y encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

Respuesta correcta: A) El compilador genera automáticamente un constructor por defecto sin argumentos

Si no se declara ningún constructor, el compilador provee uno por defecto (sin parámetros) que llama implícitamente a super().

2. ¿Qué significa que un atributo sea 'static'?

Respuesta correcta: A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual

Un miembro static pertenece a la clase, no a instancias individuales; existe una sola copia compartida por todos los objetos.

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

Respuesta correcta: A) Impedir que la variable sea reasignada después de su primera asignación

'final' en una variable impide reasignación tras la inicialización. En una clase impide herencia; en un método impide override.

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

Respuesta correcta: A) No, una clase final no puede tener subclases

'final' en una clase prohíbe que otras clases la extiendan, por ejemplo java.lang.String es final.

Herencia y polimorfismo

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

Respuesta correcta: A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación

El overriding ocurre cuando una subclase redefine un método heredado con la misma firma; el overloading es tener varios métodos con el mismo nombre pero distinta lista de parámetros.

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

Respuesta correcta: A) super

'super' referencia la superclase; super() invoca su constructor y super.metodo() invoca su implementación de un método.

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }
```

```
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }
```

```
Animal a = new Perro();
```

```
a.sonido();
```

¿Qué se imprime?

Respuesta correcta: A) Guau

El polimorfismo en tiempo de ejecución (dynamic dispatch) hace que se ejecute la versión sobrescrita del objeto real (Perro), no la del tipo de referencia declarado (Animal).

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

Respuesta correcta: A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero Sí permite implementar múltiples interfaces

Una clase solo puede extender una única superclase (extends), pero puede implementar múltiples interfaces (implements A, B, C).

Interfaces y clases abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

Respuesta correcta: A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir

Los métodos default tienen cuerpo dentro de la interfaz, permitiendo evolucionar interfaces sin romper implementaciones existentes.

2. ¿Puede una clase abstracta tener constructores?

Respuesta correcta: A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas

Las clases abstractas pueden tener constructores, campos y métodos concretos; se invocan mediante `super()` al construir una subclase concreta.

3. ¿Qué es una clase anónima en Java?

Respuesta correcta: A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual

Una clase anónima combina la declaración de una subclase (o implementación de interfaz) y su instanciación en una sola expresión, sin nombre propio.

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

Respuesta correcta: A) Exactamente uno (además de métodos default/static)

Una interfaz funcional (anotada opcionalmente con `@FunctionalInterface`) debe declarar exactamente un método abstracto, lo que permite implementarla con una expresión lambda.

Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

Respuesta correcta: A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces

Los enums son clases especiales; pueden tener campos, constructores (implícitamente private) y métodos, además de implementar interfaces y sobrescribir métodos por constante.

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

Respuesta correcta: A) ordinal()

`ordinal()` devuelve la posición de la constante según el orden de declaración, comenzando en 0. `name()` devuelve su nombre como String.

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

Respuesta correcta: A) Sí, solo se usa el nombre de la constante (ej. case LUNES ->)

En un switch sobre un enum, los case usan directamente el nombre de la constante sin calificar con el nombre de la clase.

Records

1. ¿Qué genera automáticamente el compilador al declarar un record?

Respuesta correcta: A) Constructor canónico, métodos de acceso (accessors) con el nombre del componente, equals(), hashCode() y toString()

Un record genera automáticamente: constructor canónico, accessors (con el mismo nombre del campo, sin prefijo 'get'), equals(), hashCode() y toString(). No genera setters porque los records son inmutables.

2. ¿Son mutables los componentes de un record?

Respuesta correcta: A) No, todos los campos de un record son implícitamente final e inmutables

Los records son inmutables por diseño: sus campos son implícitamente private final, sin setters generados.

3. ¿Qué es un 'constructor compacto' en un record?

Respuesta correcta: A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita

El constructor compacto se escribe como 'public NombreRecord { ... validaciones ... }' sin paréntesis de parámetros; el compilador asigna los campos automáticamente al final.

Sealed classes y pattern matching

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

Respuesta correcta: A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'

'sealed' junto con 'permits' limita el conjunto de subtipos autorizados, permitiendo un control exhaustivo (útil junto a pattern matching en switch).

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

Respuesta correcta: A) final, sealed o non-sealed

Cada subclase permitida de un tipo sealed debe declararse explícitamente como final, sealed (para seguir restringiendo) o non-sealed (para abrir la jerarquía de nuevo).

3. ¿Qué permite el pattern matching for instanceof (Java 16+)?

Respuesta correcta: A) Combinar la comprobación instanceof con la asignación a una variable en una sola expresión, evitando el cast manual: if (obj instanceof String s) {...}

Pattern matching for instanceof permite escribir 'if (obj instanceof String s)' y usar 's' ya tipado y con cast implícito dentro del bloque, sin necesidad de castear manualmente.

4. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

Respuesta correcta: A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. case Punto(int x, int y) -> ...)

Los record patterns (Java 21) permiten deconstruir un record en el propio patrón del case, extrayendo sus componentes como variables locales.

Generics y covarianza

1. ¿Qué problema resuelven los Generics en Java?

Respuesta correcta: A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando ClassCastException en tiempo de ejecución y castings manuales

Los generics permiten parametrizar tipos (List<String>, por ejemplo), detectando errores de tipo en compilación en vez de en runtime.

2. ¿Qué significa el comodín '? extends Number' en una declaración de generics?

Respuesta correcta: A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')

'? extends Number' es un wildcard covariante: acepta Number o subtipos. Sigue el principio PECS (Producer Extends, Consumer Super); con extends solo puedes leer, no añadir (excepto null).

3. ¿Por qué `List<String>` NO es un subtipo de `List<Object>` en Java (a diferencia de los arrays)?

Respuesta correcta: A) Porque los generics son invariantes: aunque `String` sea subtipo de `Object`, `List<String>` y `List<Object>` son tipos distintos sin relación de subtipo entre sí

A diferencia de los arrays (covariantes, lo que puede causar `ArrayStoreException` en runtime), los generics son invariantes por diseño para mantener la seguridad de tipos en compilación.

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

Respuesta correcta: A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u `Object`) en el bytecode

Debido a type erasure, la JVM no conserva los parámetros de tipo en tiempo de ejecución; `List<String>` y `List<Integer>` comparten la misma clase `List` en bytecode.

Comparable y Comparator

1. ¿Cuál es la diferencia principal entre `Comparable` y `Comparator`?

Respuesta correcta: A) `Comparable` se implementa en la propia clase para definir su orden natural (`compareTo`); `Comparator` se define externamente y permite múltiples estrategias de ordenación (`compare`)

`Comparable` define el orden natural de un objeto mediante `compareTo()`; `Comparator` es una interfaz separada para definir órdenes alternativos mediante `compare(a,b)`, útil para múltiples criterios.

2. ¿Qué valor devuelve `compareTo()` si el objeto actual es 'mayor' que el que se compara?

Respuesta correcta: A) Un valor positivo

Por convención: negativo si el objeto es menor, 0 si es igual, positivo si es mayor. No se garantiza un valor exacto, solo el signo.

3. ¿Cómo se encadenan múltiples criterios de comparación usando `Comparator` (por ejemplo, por apellido y luego por nombre)?

Respuesta correcta: A) `Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)`

El método `thenComparing()` permite encadenar criterios de desempate adicionales tras el comparador principal.

Lambdas e interfaces funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

Respuesta correcta: A) `(a, b) -> a + b`

La sintaxis de lambdas en Java es (parámetros) -> expresión_o_bloque. No se usa '=>' como en JavaScript.

2. ¿Qué interfaz funcional del paquete `java.util.function` representa una operación que recibe un argumento y devuelve un boolean?

Respuesta correcta: A) `Predicate<T>`

`Predicate<T>` tiene el método `test(T t)` que devuelve boolean, típicamente usado para filtrar (`Stream.filter`).

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

Respuesta correcta: A) `Supplier<T>`

`Supplier<T>` tiene el método `get()` sin parámetros que produce/devuelve un valor de tipo T, útil para provisión perezosa de valores.

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

Respuesta correcta: A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas

Las lambdas solo pueden capturar variables locales 'effectively final' (que no cambian tras su inicialización); intentar reasignarlas produce error de compilación.

Method references

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

Respuesta correcta: A) Clase::metodoEstatico

Referencia a método estático: Clase::metodo. Ejemplo: Integer.parseInt equivale a s -> Integer.parseInt(s).

2. ¿Qué representa la referencia a método String::toUpperCase usada como Function<String,String>?

Respuesta correcta: A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a s -> s.toUpperCase()

Cuando se usa Clase::metodoDeInstancia sin especificar objeto, el primer parámetro de la interfaz funcional se convierte en el receptor de la llamada: equivale a s -> s.toUpperCase().

3. ¿Cómo se escribe una referencia a constructor para crear instancias de ArrayList mediante un Supplier?

Respuesta correcta: A) ArrayList::new

Clase::new es la sintaxis de referencia a constructor, usable como Supplier<ArrayList> u otra interfaz funcional compatible con la firma del constructor.

Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

Respuesta correcta: A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez

Los streams no almacenan elementos; describen operaciones sobre una fuente de datos, se evalúan de forma perezosa y son de un solo uso.

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

Respuesta correcta: A) collect()

collect(), forEach(), count(), reduce() son operaciones terminales que disparan el procesamiento. filter, map, sorted, distinct son intermedias y perezosas.

3. ¿Qué hace el siguiente código?

```
List<String> nombres = List.of("Ana","Luis","Eva");  
nombres.stream().filter(n -> n.length()>3).forEach(System.out::println);
```

Respuesta correcta: A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres

filter conserva solo elementos cuyo length() sea mayor a 3. 'Ana'(3) y 'Eva'(3) no cumplen; solo 'Luis'(4) pasa el filtro.

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

Respuesta correcta: A) java.util.stream.Collectors

La clase utilitaria Collectors provee fábricas de Collector como toList(), toSet(), toMap(), joining(), groupingBy(), etc.

5. ¿Qué hace Collectors.groupingBy()?

Respuesta correcta: A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un Map<clave, List<elementos>>

groupingBy(funcionClasificadora) agrupa los elementos según el resultado de esa función, generando un Map donde cada clave mapea a una lista (u otro collector downstream) de elementos.

6. ¿Cuál es la diferencia entre map() y flatMap() en Streams?

Respuesta correcta: A) map() transforma cada elemento en otro elemento uno a uno; flatMap() transforma cada elemento en un stream y luego 'aplana' todos esos streams en uno solo

flatMap se usa cuando cada elemento produce un Stream (por ejemplo, una lista de listas), y combina/aplana todos esos streams internos en un único Stream resultante.

Optional

1. ¿Cuál es el propósito principal de la clase Optional<T>?

Respuesta correcta: A) Representar explícitamente la posible ausencia de un valor, evitando NullPointerException y forzando un manejo explícito del caso 'vacío'

Optional<T> es un contenedor que puede o no tener un valor, comunicando explícitamente en la firma del método que el resultado puede estar ausente.

2. ¿Qué hace Optional.ofNullable(valor) si 'valor' es null?

Respuesta correcta: A) Devuelve un Optional vacío (Optional.empty()) sin lanzar excepción

A diferencia de Optional.of() (que lanza NPE si el argumento es null), ofNullable() acepta null y produce un Optional vacío en ese caso.

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un Optional si está vacío?

Respuesta correcta: A) optional.orElse(valorPorDefecto)

orElse(valorDefault) devuelve el valor contenido si existe, o el valor por defecto indicado si el Optional está vacío. También existe orElseGet() y orElseThrow().

Collections Framework

1. ¿Cuál es la diferencia principal entre List, Set y Map en el Collections Framework?

Respuesta correcta: A) List permite duplicados y mantiene orden de inserción; Set no permite duplicados; Map almacena pares clave-valor

List: colección ordenada indexada con duplicados permitidos. Set: no permite elementos duplicados. Map: asocia claves únicas a valores (no extiende Collection).

2. ¿Qué implementación de List garantiza acceso rápido O(1) por índice, pero inserciones/eliminaciones costosas en el medio?

Respuesta correcta: A) ArrayList

ArrayList se respalda en un array, por lo que el acceso por índice es O(1), pero insertar/eliminar en medio requiere desplazar elementos. LinkedList es más eficiente en inserciones/eliminaciones.

3. ¿Qué característica distingue a un TreeMap de un HashMap?

Respuesta correcta: A) TreeMap mantiene sus claves ordenadas (orden natural o Comparator); HashMap no garantiza ningún orden

TreeMap implementa NavigableMap y mantiene las claves ordenadas; HashMap ofrece acceso O(1) promedio pero sin orden garantizado.

4. ¿Qué devuelve List.of(1,2,3) en cuanto a mutabilidad?

Respuesta correcta: A) Una lista inmutable; intentar añadir o eliminar elementos lanza UnsupportedOperationException

Los métodos de fábrica List.of(), Set.of(), Map.of() (Java 9+) devuelven colecciones inmutables; cualquier intento de modificarlas lanza UnsupportedOperationException.

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un HashMap de forma consistente?

Respuesta correcta: A) equals() y hashCode() (contrato de Object)

HashMap usa hashCode() para ubicar el bucket y equals() para verificar igualdad de claves; si no se sobrescriben correctamente y de forma consistente, el Map puede comportarse mal.

Concurrencia y virtual threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

Respuesta correcta: A) Callable<V>

Callable<V> define call() que devuelve un valor V y puede lanzar excepciones checked; Runnable define run() sin retorno y sin poder lanzar checked exceptions.

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

Respuesta correcta: A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida

ExecutorService (java.util.concurrent) administra un conjunto de hilos, permitiendo enviar tareas sin gestionar manualmente la creación y destrucción de cada Thread.

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

Respuesta correcta: A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O

Los virtual threads (Project Loom, estándar desde Java 21) son gestionados por la JVM en lugar de mapearse 1:1 a hilos del SO, permitiendo alta concurrencia con muy bajo overhead.

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

Respuesta correcta: A) synchronized

'synchronized' aplicado a un método o bloque asegura exclusión mutua: solo un hilo puede ejecutar esa sección crítica sobre el mismo monitor a la vez.

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

Respuesta correcta: A) Executors.newVirtualThreadPerTaskExecutor()

Executors.newVirtualThreadPerTaskExecutor() crea un ExecutorService que lanza una nueva virtual thread por cada tarea enviada.

Manejo de excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

Respuesta correcta: A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden Exception); las unchecked no obligan a manejo explícito (extienden RuntimeException)

Las excepciones checked (como IOException) obligan al compilador a exigir manejo o declaración con throws; las unchecked (RuntimeException y subclasses) no tienen esa obligación.

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

Respuesta correcta: A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo System.exit() o error grave de JVM)

El bloque finally se ejecuta siempre tras try/catch, independientemente de si hubo excepción o un return, salvo casos extremos como System.exit().

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

Respuesta correcta: A) Cierra automáticamente los recursos que implementan AutoCloseable al finalizar el bloque try, sin necesidad de un finally explícito

try-with-resources cierra automáticamente cada recurso declarado (AutoCloseable/Closeable) al salir del bloque, en orden inverso a su declaración, incluso si ocurre una excepción.

4. ¿Qué clase es la superclase común de Exception y Error?

Respuesta correcta: A) Throwable

java.lang.Throwable es la raíz de la jerarquía; de ella derivan Exception (condiciones recuperables) y Error (problemas graves, normalmente no manejables).

BigDecimal y precisión numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

Respuesta correcta: A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria

double/float usan representación binaria IEEE 754 que no puede representar exactamente muchos decimales, causando errores acumulativos; BigDecimal evita esto mediante representación decimal exacta.

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

Respuesta correcta: A) new BigDecimal("19.99") usando un String, para evitar imprecisión del double

Construir BigDecimal a partir de un double literal puede arrastrar la imprecisión binaria de ese double; usar el constructor con String es más seguro.

3. ¿Por qué comparar dos BigDecimal con equals() puede dar resultados inesperados, por ejemplo entre new BigDecimal("2.0") y new BigDecimal("2.00")?

Respuesta correcta: A) Porque equals() considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar compareTo()

equals() en BigDecimal compara valor Y escala; '2.0' y '2.00' son matemáticamente iguales pero tienen distinta escala, por lo que equals() devuelve false. compareTo() sí compara solo el valor numérico.

Internacionalización (Locale, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

Respuesta correcta: A) java.util.Locale

Locale encapsula idioma y país/región, usado por clases como NumberFormat, DateFormat y ResourceBundle para adaptar la salida.

2. ¿Cómo se cargan mensajes traducidos según el idioma usando ResourceBundle?

Respuesta correcta: A) ResourceBundle.getBundle("nombreBase", locale) busca automáticamente el archivo .properties más específico que coincida con el Locale dado, con fallback al bundle por defecto

ResourceBundle.getBundle busca archivos como mensajes_es_MX.properties, mensajes_es.properties, hasta mensajes.properties (default), siguiendo el algoritmo de resolución de Locale.

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un Locale específico?

Respuesta correcta: A) java.text.NumberFormat

NumberFormat.getInstance(locale) o NumberFormat.getCurrencyInstance(locale) formatean números respetando las convenciones del Locale indicado.

Patrones de diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

Respuesta correcta: A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa

Strategy define una interfaz común para varios algoritmos intercambiables; el contexto delega en la estrategia concreta, permitiendo cambiarla en tiempo de ejecución sin modificar el cliente.

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

Respuesta correcta: A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia

Si la interfaz Strategy es funcional, se puede pasar una lambda o method reference como estrategia en lugar de crear una clase concreta por cada algoritmo.

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

Respuesta correcta: A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)

Strategy favorece el principio Open/Closed: se pueden añadir nuevas estrategias creando nuevas implementaciones, sin modificar el código cliente existente.